Mini project report on

GITHUB REPOSITORY INTELLIGENCE & DEVELOPER ECOSYSTEM MAPPER

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE23CS351A – DBMS Project

Submitted by:

PRATHEEK P SHETTIGAR	**PES2UG24CS650**
AKASHA	**PES2UG25CS803**

under the guidance of

Dr. Shilpa S

Assistant Professor

PES University

AUG - DEC 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

GITHUB REPOSITORY INTELLIGENCE & DEVELOPER ECOSYSTEM MAPPER

is a bonafide work carried out by

PRATHEEK P SHETTIGAR	**PES2UG24CS650**
AKASHA	**PES2UG25CS803**

In partial fulfilment for the completion of fifth semester DBMS Project (UE24CSS351A) in the Program of Study -Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period AUG. 2026 – DEC. 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 5th semester academic requirements in respect of project work.

Signature
Dr. Shilpa S
Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled **Disasters Data Management System** has been carried out by us under the guidance of **Dr. Shilpa S, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester AUG – DEC 2026.

PRATHEEK P SHETTIGAR	**PES2UG24CS650**
AKASHA	**PES2UG25CS803**

ACKNOWLEDGEMENT

I would like to express my gratitude to Dr. Shilpa S, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE23CS351A - DBMS Project.

I take this opportunity to thank Dr. Sandesh B J, C, Professor, ChairPerson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to me various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

GitHub Repository Intelligence & Developer Ecosystem Mapper is a database-oriented mini project designed to represent and query information associated with software repositories and their developer ecosystem. The database model captures users, accounts, repositories, forks, commits, tools, files, file versions, releases, release statistics, issues, pull requests, reviews, stars, follows and repository collaborators.

The project uses a relational database structure with primary keys and foreign-key relationships to connect repository activity with the users and artefacts involved. The submitted SQL work demonstrates data insertion, update and delete operations; basic retrieval; aggregate functions; GROUP BY and HAVING; joins; nested and correlated subqueries; UNION; views; indexes; transactions; triggers; stored functions; stored procedures; common table expressions (CTEs); and window functions.

The implementation is illustrated using sample repository records such as Smart-City-IoT, DBMS-Lab, ML-Toolkit, Web-Portal and Legacy-App. The report documents the conceptual model, relational mapping, table definitions and selected query executions. The supplied project materials focus on the database layer; a separate front-end implementation is not included in the materials reviewed.

TABLE OF CONTENTS

Chapter No.	Title	Page No.
1	INTRODUCTION	8
2	PROBLEM DEFINITION	9
3	ER MODEL	10
4	ER TO RELATIONAL MAPPING	12
5	DDL STATEMENTS	14
6	DML STATEMENTS	15
7	QUERIES	19
7.1	Simple query with aggregate functions and GROUP BY	19
7.2	UPDATE operation	21
7.3	DELETE operation	22
7.4	Correlated query	
7.5	Nested query	27
8	STORED PROCEDURES, FUNCTIONS AND TRIGGERS	29
8.1	Stored function	29
8.2	Stored procedure	30
8.3	Trigger	29
9	FRONT-END DEVELOPMENT	32
10	MongoDB DATABASE	38
REFERENCES	REFERENCES	40

LIST OF TABLES

Table No.	Title	Page No.
1.1	**Main entities in the project database**	11
4.1	**Relational tables and their primary keys**	13
4.2	**Important foreign-key relationships**	13
5.1	**Summary of database tables**	14
7.1	**SQL query examples for repository and developer data**	19
10.1	**MongoDB collections and their roles in Orbit**	39

LIST OF FIGURES

Figure No.	*Figure Description*	Source / Reference
3.1	**Entity–Relationship (ER) Diagram**	Review 1, Page 2
4.1	**Relational Schema Diagram**	Review 1, Page 3
5.1	***Table Creation Evidence***	Review 1, Pages 4–10
6.1	**Data Insertion Evidence**	Review 2, Pages 1–4
7.1	**Query Execution Evidence**	Review 2, Selected Pages
8.1	**Trigger, Function and Procedure Execution Evidence**	Review 2, Pages 18–20

1. INTRODUCTION

Modern software development platforms contain interconnected information about repositories, developers, code changes, issues, pull requests, reviews, releases and collaboration. Analysing these records independently makes it difficult to understand repository activity and the relationships among contributors and project artefacts.

The GitHub Repository Intelligence & Developer Ecosystem Mapper project models these activities in a relational database. The design organises information into related tables and supports retrieval of repository details, contributor activity, file history, release information and collaboration relationships.

1.1 Objectives

- Represent users and their repository-related accounts.
- Store repository metadata, programming language, visibility, archive status, stars and forks.
- Record commits, files, file versions, issues, pull requests and reviews.
- Represent stars, follows and collaborator relationships between users and repositories.
- Demonstrate SQL operations and database features for analysing the stored information.

1.2 Scope

The project includes a web application named Orbit and its database layer. The application connects to GitHub through OAuth, retrieves repository and developer activity data, and presents repository management, analytics, AI-assisted explanations and ecosystem mapping. The submitted DBMS coursework demonstrates relational modelling and SQL operations, while the running Orbit website uses MongoDB document collections for application accounts, repository snapshots, settings and audit data.

1.3 Technologies represented

- The academic SQL scripts use MySQL to demonstrate the relational schema and required DBMS operations. Orbit's website backend uses MongoDB, a NoSQL document database, for account records, linked provider identities, repositories, repository snapshots, projects, settings and audit events.
- The relational coursework demonstrates DDL, DML, joins, aggregate queries, nested and correlated subqueries, views, indexes, transactions, triggers, stored routines, CTEs and window functions. The website uses MongoDB collections and document operations for its live application data; these are distinct database layers and are described separately in this report.

2. PROBLEM DEFINITION

Repository information is naturally distributed across multiple related concepts. A repository can contain files and commits, receive stars and forks, publish releases, and maintain issues and pull requests. Users may contribute commits, review pull requests, follow other users and collaborate on repositories. Without a structured database, storing and retrieving these related records consistently becomes difficult.

2.1 Problem statement

Design a relational database that organises repository and developer ecosystem information, maintains relationships among the entities, and supports SQL-based querying and analysis of repository activity.

2.2 Proposed database solution

The project uses a set of related tables connected through primary and foreign keys. Repository-specific records reference Repositories; user activities reference Users; file history references Files; review records reference Pull_Requests; and release statistics reference Releases. SQL queries combine these tables to produce repository and developer activity information.

2.3 Expected outcomes

- Consistent storage of repository and user information.
- Traceable relationships among commits, files, versions and repositories.
- Retrieval of repository statistics and activity using SQL.
- Demonstration of data integrity and database programmability features.

3. ER MODEL

The ER model in the submitted Review 1 document identifies the major entities and their relationships. It includes user and account information, repositories, commits, tools, files and file versions, releases and release statistics, issues, pull requests, reviews, stars, follows and collaborators.

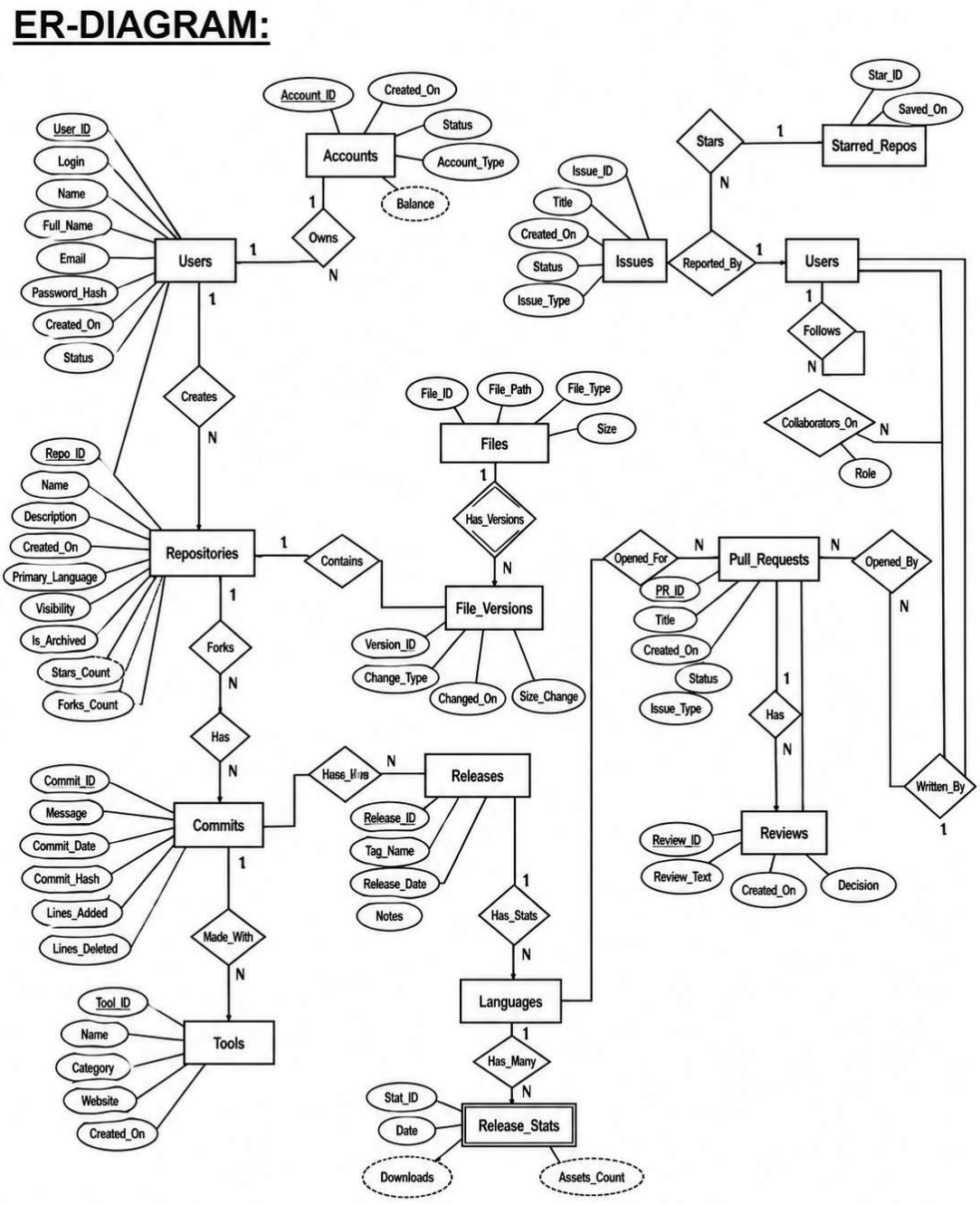


Figure 3.1: Entity–Relationship diagram supplied in DBMS Review 1.

3.1 Main entities

Entity	Purpose
Users	Stores user identity, login, name, email, password hash, role, creation date and status.
Accounts	Stores account details associated with a user.
Repositories	Stores repository name, description, language, visibility, archive flag, stars and forks.
Forks and Stars	Record users' fork and star relationships with repositories.
Commits	Stores commit messages, dates, hashes and code-change counts.
Files and File_Versions	Represent repository files and their recorded changes over time.
Issues, Pull_Requests and Reviews	Represent issue tracking, proposed changes and review decisions.
Releases and Releases_Stats	Represent tagged releases and their download and asset statistics.
Follows and Collaborators	Represent user-to-user following and repository collaboration.

3.2 Relationship overview

The ER diagram shows one-to-many relationships such as a repository having multiple commits, files, forks, issues, pull requests and releases. File versions are associated with files. Reviews are associated with pull requests and users. Stars and follows represent relationship records rather than simple descriptive attributes.

4. ER TO RELATIONAL MAPPING

The ER model is represented as relational tables. Each entity becomes a table with a primary key. Relationships are represented through foreign keys or associative tables, depending on the relationship.

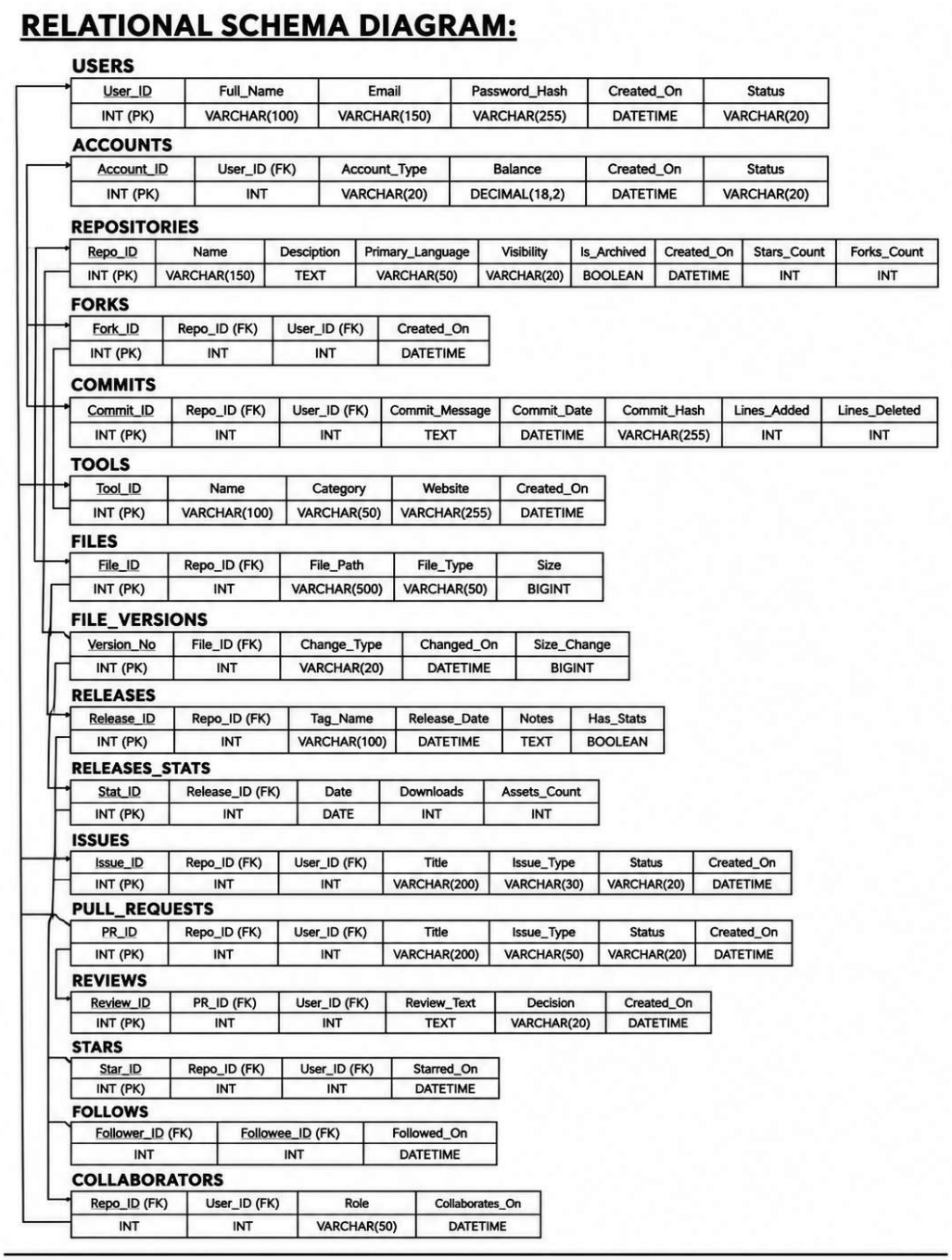


Figure 4.1: Relational schema diagram supplied in DBMS Review 1.

4.1 Steps of algorithm for chosen problem

1. Identify the entity sets in the ER model, including Users, Repositories, Commits, Files, Issues and Pull_Requests.

2. Create one relation for each strong entity and select its primary key.

3. Represent one-to-many relationships by placing the primary key of the parent relation as a foreign key in the child relation.

4. Represent user-to-repository and user-to-user relationships using relationship tables such as Stars, Collaborators and Follows.

5. Represent file history using File_Versions with a reference to Files.

6. Represent review and release-statistics details using Reviews and Releases_Stats with references to their parent tables.

7. Map attributes to suitable SQL data types and define required constraints.

4.2 Relational tables and primary keys

Relation	Primary key shown in project schema
Users	User_ID
Accounts	Account_ID
Repositories	Repo_ID
Forks	Fork_ID
Commits	Commit_ID
Tools	Tool_ID
Files	File_ID
File_Versions	Version_No
Releases	Release_ID
Releases_Stats	Stat_ID
Issues	Issue_ID
Pull_Requests	PR_ID
Reviews	Review_ID
Stars	Star_ID
Follows	Follow_ID
Collaborators	(Repo_ID, User_ID)

4.3 Important foreign-key relationships

- Accounts.User_ID references Users.User_ID.
- Forks.Repo_ID references Repositories.Repo_ID; Forks.User_ID references Users.User_ID.
- Commits.Repo_ID references Repositories.Repo_ID; Commits.User_ID references Users.User_ID.
- Files.Repo_ID references Repositories.Repo_ID; File_Versions.File_ID references Files.File_ID.
- Releases.Repo_ID references Repositories.Repo_ID; Releases_Stats.Release_ID references Releases.Release_ID.
- Issues and Pull_Requests reference Repositories and Users; Reviews references Pull_Requests and Users.
- Stars and Collaborators reference Repositories and Users; Follows references Users for both follower and followed identities.

5. DDL STATEMENTS

Data Definition Language (DDL) statements define the database structure. The Review 1 submission contains table-creation statements and DESCRIBE output for the project relations. The schema uses integer identifiers, character fields, text, date/time values, decimal values and Boolean fields.

5.1 Table creation summary

Table	Primary key	Purpose
Users	User_ID	User profile and role information
Accounts	Account_ID	Account records linked to users
Repositories	Repo_ID	Repository metadata and counters
Forks	Fork_ID	Repository fork records
Commits	Commit_ID	Commit details and code changes
Tools	Tool_ID	Development tools
Files	File_ID	Files belonging to repositories
File_Versions	Version_No	Recorded file changes
Releases	Release_ID	Repository releases
Releases_Stats	Stat_ID	Release download and asset statistics
Issues	Issue_ID	Repository issue records
Pull_Requests	PR_ID	Proposed repository changes
Reviews	Review_ID	Pull request review decisions
Stars	Star_ID	Repository star records
Follows	Follow_ID	User following relationships
Collaborators	Repo_ID, User_ID	Repository collaboration and role

5.2 Representative DDL statement

```
CREATE TABLE Repositories (
    Repo_ID INT AUTO_INCREMENT PRIMARY KEY,
    Name VARCHAR(150) NOT NULL,
    Description TEXT,
    Primary_Language VARCHAR(50) NOT NULL DEFAULT 'public',
    Visibility VARCHAR(20) NOT NULL DEFAULT 'public',
    Is_Archived BOOLEAN NOT NULL DEFAULT FALSE,
    Created_On TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    Stars_Count INT NOT NULL DEFAULT 0,
    Forks_Count INT NOT NULL DEFAULT 0
);
```

The complete table-creation evidence for the 16 relations is provided in the submitted Review 1 PDF. Selected screenshots are included in the evidence section of this report.

6. DML STATEMENTS

Data Manipulation Language (DML) statements insert, update and delete records in the database. The Review 2 SQL script inserts sample data for users, accounts, repositories, forks, commits, tools, files, file versions, releases, release statistics, issues, pull requests, reviews, stars, follows and collaborators.

PART 1: INSERT DATA :

```
mysql> USE github_mapper;
Database changed
mysql> INSERT INTO Users
    -> (User_ID, Login, Name, Full_Name, Email, Password_Hash, Role, Status)
    -> VALUES
    -> (1, 'pratheek', 'Pratheek', 'Pratheek P Shettigar',
    ->  'pratheek@example.com', 'hash_001', 'Developer', 'ACTIVE'),
    -> (2, 'akasha', 'Akasha', 'Akasha',
    ->  'akasha@example.com', 'hash_002', 'Reviewer', 'ACTIVE'),
    -> (3, 'rahuldev', 'Rahul', 'Rahul Kumar',
    ->  'rahul@example.com', 'hash_003', 'Developer', 'ACTIVE'),
    -> (4, 'megha', 'Megha', 'Megha Rao',
    ->  'megha@example.com', 'hash_004', 'Maintainer', 'ACTIVE'),
    -> (5, 'admin01', 'Admin', 'System Admin',
    ->  'admin@example.com', 'hash_005', 'Admin', 'ACTIVE');
Query OK, 5 rows affected (0.03 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Accounts
    -> (Account_ID, User_ID, Account_Type, Balance, Status)
    -> VALUES
    -> (1, 1, 'GitHub', 1000.00, 'ACTIVE'),
    -> (2, 2, 'GitHub', 1500.00, 'ACTIVE'),
    -> (3, 3, 'GitHub', 800.00, 'ACTIVE');
Query OK, 3 rows affected (0.04 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Repositories
    -> (Repo_ID, Name, Description, Primary_Language, Visibility,
    ->  Is_Archived, Stars_Count, Forks_Count)
    -> VALUES
    -> (101, 'Smart-City-IoT', 'IoT analytics project', 'C++',
    ->  'public', FALSE, 25, 7),
    -> (102, 'DBMS-Lab', 'Database management exercises', 'SQL',
    ->  'public', FALSE, 18, 4),
    -> (103, 'ML-Toolkit', 'Machine learning utilities', 'Python',
    ->  'public', FALSE, 42, 11),
    -> (104, 'Web-Portal', 'Developer web portal', 'JavaScript',
    ->  'private', FALSE, 9, 2),
    -> (105, 'Legacy-App', 'Archived legacy application', 'Java',
    ->  'public', TRUE, 5, 1);
Query OK, 5 rows affected (0.06 sec)
Records: 5  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Forks
    -> (Fork_ID, Repo_ID, User_ID, Created_On)
    -> VALUES
    -> (1, 101, 2, '2026-09-10 10:00:00'),
    -> (2, 101, 3, '2026-09-11 11:00:00'),
    -> (3, 102, 1, '2026-09-12 12:00:00'),
    -> (4, 103, 4, '2026-09-13 13:00:00');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

Figure 6.1: Insert statements and execution output, Review 2 page 1.

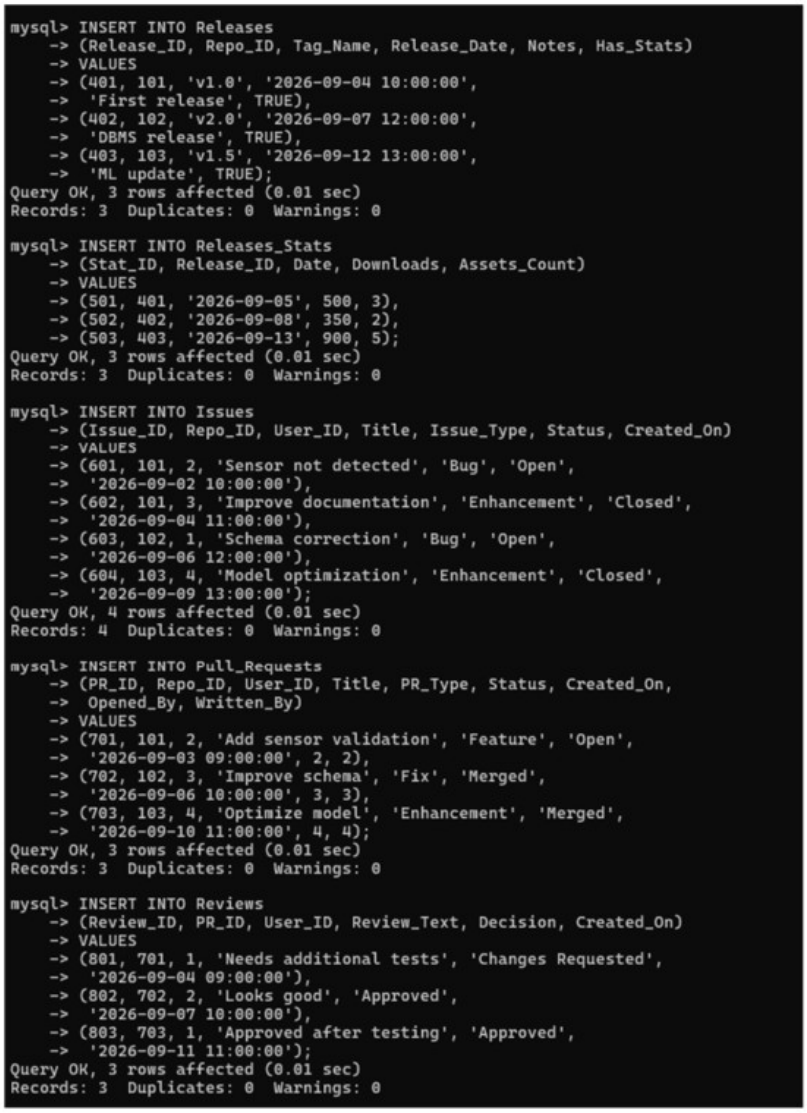
```
mysql> INSERT INTO Commits
    -> (Commit_ID, Repo_ID, User_ID, Commit_Message, Commit_Date,
    ->  Commit_Hash, Lines_Added, Lines_Deleted)
    -> VALUES
    -> (1001, 101, 1, 'Add sensor module', '2026-09-01 09:00:00',
    ->  'hash101a', 120, 20),
    -> (1002, 101, 3, 'Fix sensor bug', '2026-09-03 10:30:00',
    ->  'hash101b', 80, 15),
    -> (1003, 102, 1, 'Create database schema', '2026-09-05 14:00:00',
    ->  'hash102a', 200, 30),
    -> (1004, 102, 2, 'Add SQL queries', '2026-09-06 15:00:00',
    ->  'hash102b', 150, 10),
    -> (1005, 103, 3, 'Add regression model', '2026-09-08 11:00:00',
    ->  'hash103a', 300, 50),
    -> (1006, 103, 4, 'Improve preprocessing', '2026-09-09 12:00:00',
    ->  'hash103b', 180, 25),
    -> (1007, 104, 2, 'Update login page', '2026-09-10 16:00:00',
    ->  'hash104a', 90, 12),
    -> (1008, 105, 4, 'Legacy maintenance', '2026-09-11 17:00:00',
    ->  'hash105a', 40, 8);
Query OK, 8 rows affected (0.01 sec)
Records: 8  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Tools
    -> (Tool_ID, Name, Category, Website)
    -> VALUES
    -> (201, 'Git', 'Version Control', 'https://git-scm.com'),
    -> (202, 'Docker', 'Containerization', 'https://docker.com'),
    -> (203, 'Jenkins', 'CI/CD', 'https://jenkins.io');
Query OK, 3 rows affected (0.02 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Files
    -> (File_ID, Repo_ID, File_Path, File_Type, Size)
    -> VALUES
    -> (301, 101, 'src/sensor.cpp', 'cpp', 12000),
    -> (302, 101, 'README.md', 'markdown', 5000),
    -> (303, 102, 'schema.sql', 'sql', 9000),
    -> (304, 103, 'model.py', 'python', 15000),
    -> (305, 104, 'src/login.js', 'javascript', 11000),
    -> (306, 105, 'Main.java', 'java', 8000);
Query OK, 6 rows affected (0.01 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> INSERT INTO File_Versions
    -> (Version_No, File_ID, Change_Type, Changed_On, Size_Change)
    -> VALUES
    -> (1, 301, 'Added', '2026-09-01 09:00:00', 12000),
    -> (2, 301, 'Modified', '2026-09-03 10:30:00', 1500),
    -> (3, 303, 'Added', '2026-09-05 14:00:00', 9000),
    -> (4, 304, 'Added', '2026-09-08 11:00:00', 15000),
    -> (5, 305, 'Modified', '2026-09-10 16:00:00', 1000);
Query OK, 5 rows affected (0.04 sec)
Records: 5  Duplicates: 0  Warnings: 0
```

Figure 6.2: Insert statements and execution output, Review 2 page 2.



```
mysql> INSERT INTO Releases
    -> (Release_ID, Repo_ID, Tag_Name, Release_Date, Notes, Has_Stats)
    -> VALUES
    -> (401, 101, 'v1.0', '2026-09-04 10:00:00',
    ->  'First release', TRUE),
    -> (402, 102, 'v2.0', '2026-09-07 12:00:00',
    ->  'DBMS release', TRUE),
    -> (403, 103, 'v1.5', '2026-09-12 13:00:00',
    ->  'ML update', TRUE);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

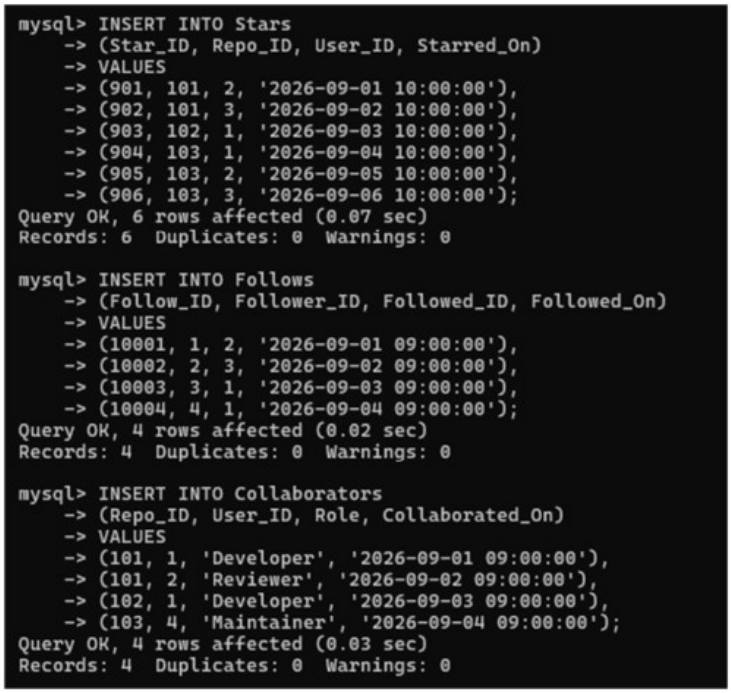
mysql> INSERT INTO Releases_Stats
    -> (Stat_ID, Release_ID, Date, Downloads, Assets_Count)
    -> VALUES
    -> (501, 401, '2026-09-05', 500, 3),
    -> (502, 402, '2026-09-08', 350, 2),
    -> (503, 403, '2026-09-13', 900, 5);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Issues
    -> (Issue_ID, Repo_ID, User_ID, Title, Issue_Type, Status, Created_On)
    -> VALUES
    -> (601, 101, 2, 'Sensor not detected', 'Bug', 'Open',
    ->  '2026-09-02 10:00:00'),
    -> (602, 101, 3, 'Improve documentation', 'Enhancement', 'Closed',
    ->  '2026-09-04 11:00:00'),
    -> (603, 102, 1, 'Schema correction', 'Bug', 'Open',
    ->  '2026-09-06 12:00:00'),
    -> (604, 103, 4, 'Model optimization', 'Enhancement', 'Closed',
    ->  '2026-09-09 13:00:00');
Query OK, 4 rows affected (0.01 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Pull_Requests
    -> (PR_ID, Repo_ID, User_ID, Title, PR_Type, Status, Created_On,
    ->  Opened_By, Written_By)
    -> VALUES
    -> (701, 101, 2, 'Add sensor validation', 'Feature', 'Open',
    ->  '2026-09-03 09:00:00', 2, 2),
    -> (702, 102, 3, 'Improve schema', 'Fix', 'Merged',
    ->  '2026-09-06 10:00:00', 3, 3),
    -> (703, 103, 4, 'Optimize model', 'Enhancement', 'Merged',
    ->  '2026-09-10 11:00:00', 4, 4);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Reviews
    -> (Review_ID, PR_ID, User_ID, Review_Text, Decision, Created_On)
    -> VALUES
    -> (801, 701, 1, 'Needs additional tests', 'Changes Requested',
    ->  '2026-09-04 09:00:00'),
    -> (802, 702, 2, 'Looks good', 'Approved',
    ->  '2026-09-07 10:00:00'),
    -> (803, 703, 1, 'Approved after testing', 'Approved',
    ->  '2026-09-11 11:00:00');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0
```

Figure 6.3: Insert statements and execution output, Review 2 page 3.



```
mysql> INSERT INTO Stars
    -> (Star_ID, Repo_ID, User_ID, Starred_On)
    -> VALUES
    -> (901, 101, 2, '2026-09-01 10:00:00'),
    -> (902, 101, 3, '2026-09-02 10:00:00'),
    -> (903, 102, 1, '2026-09-03 10:00:00'),
    -> (904, 103, 1, '2026-09-04 10:00:00'),
    -> (905, 103, 2, '2026-09-05 10:00:00'),
    -> (906, 103, 3, '2026-09-06 10:00:00');
Query OK, 6 rows affected (0.07 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Follows
    -> (Follow_ID, Follower_ID, Followed_ID, Followed_On)
    -> VALUES
    -> (10001, 1, 2, '2026-09-01 09:00:00'),
    -> (10002, 2, 3, '2026-09-02 09:00:00'),
    -> (10003, 3, 1, '2026-09-03 09:00:00'),
    -> (10004, 4, 1, '2026-09-04 09:00:00');
Query OK, 4 rows affected (0.02 sec)
Records: 4  Duplicates: 0  Warnings: 0

mysql> INSERT INTO Collaborators
    -> (Repo_ID, User_ID, Role, Collaborated_On)
    -> VALUES
    -> (101, 1, 'Developer', '2026-09-01 09:00:00'),
    -> (101, 2, 'Reviewer', '2026-09-02 09:00:00'),
    -> (102, 1, 'Developer', '2026-09-03 09:00:00'),
    -> (103, 4, 'Maintainer', '2026-09-04 09:00:00');
Query OK, 4 rows affected (0.03 sec)
Records: 4  Duplicates: 0  Warnings: 0
```

Figure 6.4: Insert statements and execution output, Review 2 page 4.

8. QUERIES

7.1 SIMPLE QUERY WITH GROUP BY AND AGGREGATE FUNCTIONS

2. **Average number of stars**

```
mysql> SELECT AVG(Stars_Count) AS Average_Stars
    -> FROM Repositories;
+---------------+
| Average_Stars |
+---------------+
|       20.8000 |
+---------------+
1 row in set (0.01 sec)
```

3. **Total number of stars**

```
mysql> SELECT SUM(Stars_Count) AS Total_Stars
    -> FROM Repositories;
+-------------+
| Total_Stars |
+-------------+
|         104 |
+-------------+
1 row in set (0.00 sec)
```

4. **Maximum number of forks**

```
mysql> SELECT MAX(Forks_Count) AS Maximum_Forks
    -> FROM Repositories;
+---------------+
| Maximum_Forks |
+---------------+
|            11 |
+---------------+
1 row in set (0.01 sec)
```

5. **Minimum number of stars**

```
mysql> SELECT MIN(Stars_Count) AS Minimum_Stars
    -> FROM Repositories;
+---------------+
| Minimum_Stars |
+---------------+
|             5 |
+---------------+
1 row in set (0.00 sec)
```

PART 6: GROUP BY:

1. **Number of repositories by programming language**

```
mysql> SELECT Primary_Language,
    ->        COUNT(*) AS Repository_Count
    -> FROM Repositories
    -> GROUP BY Primary_Language;
+------------------+------------------+
| Primary_Language | Repository_Count |
+------------------+------------------+
| C++              |                1 |
| SQL              |                1 |
| Python           |                1 |
| JavaScript       |                1 |
| Java             |                1 |
+------------------+------------------+
5 rows in set (0.00 sec)
```

Figure 7.9: SQL query execution evidence, Review 2 page 9.

2. **Number of repositories by visibility**

```
mysql> SELECT Visibility,
    ->        COUNT(*) AS Repository_Count
    -> FROM Repositories
    -> GROUP BY Visibility;
+------------+------------------+
| Visibility | Repository_Count |
+------------+------------------+
| public     |                4 |
| private    |                1 |
+------------+------------------+
2 rows in set (0.00 sec)
```

3. **Total stars by programming language**

```
mysql> SELECT Primary_Language,
    ->        SUM(Stars_Count) AS Total_Stars
    -> FROM Repositories
    -> GROUP BY Primary_Language;
+------------------+-------------+
| Primary_Language | Total_Stars |
+------------------+-------------+
| C++              |          30 |
| SQL              |          18 |
| Python           |          42 |
| JavaScript       |           9 |
| Java             |           5 |
+------------------+-------------+
5 rows in set (0.00 sec)
```

PART 7: HAVING:

1. **Repository languages having more than one repository**

```
mysql> SELECT Primary_Language,
    ->        COUNT(*) AS Repository_Count
    -> FROM Repositories
    -> GROUP BY Primary_Language
    -> HAVING COUNT(*) > 1;
Empty set (0.01 sec)
```

2. **Programming languages having total stars greater than 30**

```
mysql> SELECT Primary_Language,
    ->        SUM(Stars_Count) AS Total_Stars
    -> FROM Repositories
    -> GROUP BY Primary_Language
    -> HAVING SUM(Stars_Count) > 30;
+------------------+-------------+
| Primary_Language | Total_Stars |
+------------------+-------------+
| Python           |          42 |
+------------------+-------------+
1 row in set (0.00 sec)
```

Figure 7.10: SQL query execution evidence, Review 2 page 10.

7.2 UPDATE OPERATION

PART 2: UPDATE:

1. **Update repository status**

```
mysql> SELECT Repo_ID, Name, Is_Archived
    -> FROM Repositories;
+---------+----------------+-------------+
| Repo_ID | Name           | Is_Archived |
+---------+----------------+-------------+
|     101 | Smart-City-IoT |           0 |
|     102 | DBMS-Lab       |           0 |
|     103 | ML-Toolkit     |           0 |
|     104 | Web-Portal     |           0 |
|     105 | Legacy-App     |           1 |
+---------+----------------+-------------+
5 rows in set (0.00 sec)

mysql> UPDATE Repositories
    -> SET Is_Archived = TRUE
    -> WHERE Repo_ID = 105;
Query OK, 0 rows affected (0.01 sec)
Rows matched: 1  Changed: 0  Warnings: 0

mysql> SELECT Repo_ID, Name, Is_Archived
    -> FROM Repositories
    -> WHERE Repo_ID = 105;
+---------+------------+-------------+
| Repo_ID | Name       | Is_Archived |
+---------+------------+-------------+
|     105 | Legacy-App |           1 |
+---------+------------+-------------+
1 row in set (0.00 sec)
```

2. Update Pull Request status

```
mysql> SELECT PR_ID, Title, Status
    -> FROM Pull_Requests;
+-------+-----------------------+--------+
| PR_ID | Title                 | Status |
+-------+-----------------------+--------+
|   701 | Add sensor validation | Open   |
|   702 | Improve schema        | Merged |
|   703 | Optimize model        | Merged |
+-------+-----------------------+--------+
3 rows in set (0.00 sec)

mysql> UPDATE Pull_Requests
    -> SET Status = 'Merged'
    -> WHERE PR_ID = 701;
Query OK, 1 row affected (0.01 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT PR_ID, Title, Status
    -> FROM Pull_Requests
    -> WHERE PR_ID = 701;
+-------+-----------------------+--------+
| PR_ID | Title                 | Status |
+-------+-----------------------+--------+
|   701 | Add sensor validation | Merged |
+-------+-----------------------+--------+
1 row in set (0.00 sec)
```

Figure 7.5: SQL query execution evidence, Review 2 page 5.

7.3 DELETE OPERATION

3. **Update repository star count**

```
mysql> SELECT Repo_ID, Name, Stars_Count
    -> FROM Repositories;
+---------+----------------+-------------+
| Repo_ID | Name           | Stars_Count |
+---------+----------------+-------------+
|     101 | Smart-City-IoT |          25 |
|     102 | DBMS-Lab       |          18 |
|     103 | ML-Toolkit     |          42 |
|     104 | Web-Portal     |           9 |
|     105 | Legacy-App     |           5 |
+---------+----------------+-------------+
5 rows in set (0.00 sec)

mysql> UPDATE Repositories
    -> SET Stars_Count = Stars_Count + 5
    -> WHERE Repo_ID = 101;
Query OK, 1 row affected (0.02 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> SELECT Repo_ID, Name, Stars_Count
    -> FROM Repositories
    -> WHERE Repo_ID = 101;
+---------+----------------+-------------+
| Repo_ID | Name           | Stars_Count |
+---------+----------------+-------------+
|     101 | Smart-City-IoT |          30 |
+---------+----------------+-------------+
1 row in set (0.00 sec)
```

PART 3: DELETE:

1. **Delete a collaborator relationship**

```
mysql> SELECT Repo_ID, User_ID, Role
    -> FROM Collaborators;
+---------+---------+-------------+
| Repo_ID | User_ID | Role        |
+---------+---------+-------------+
|     101 |       1 | Developer   |
|     101 |       2 | Reviewer    |
|     102 |       1 | Developer   |
|     103 |       4 | Maintainer  |
+---------+---------+-------------+
4 rows in set (0.00 sec)

mysql> DELETE FROM Collaborators
    -> WHERE Repo_ID = 101
    ->   AND User_ID = 2;
Query OK, 1 row affected (0.01 sec)

mysql> SELECT Repo_ID, User_ID, Role
    -> FROM Collaborators;
+---------+---------+-------------+
| Repo_ID | User_ID | Role        |
+---------+---------+-------------+
|     101 |       1 | Developer   |
|     102 |       1 | Developer   |
|     103 |       4 | Maintainer  |
+---------+---------+-------------+
3 rows in set (0.00 sec)
```

Figure 7.6: SQL query execution evidence, Review 2 page 6.

2. **Delete a follow relationship**

```
mysql> SELECT Follow_ID, Follower_ID, Followed_ID
    -> FROM Follows;
+-----------+-------------+-------------+
| Follow_ID | Follower_ID | Followed_ID |
+-----------+-------------+-------------+
|     10001 |           1 |           2 |
|     10002 |           2 |           3 |
|     10003 |           3 |           1 |
|     10004 |           4 |           1 |
+-----------+-------------+-------------+
4 rows in set (0.00 sec)

mysql> DELETE FROM Follows
    -> WHERE Follower_ID = 1
    ->   AND Followed_ID = 2;
Query OK, 1 row affected (0.05 sec)

mysql> SELECT Follow_ID, Follower_ID, Followed_ID
    -> FROM Follows;
+-----------+-------------+-------------+
| Follow_ID | Follower_ID | Followed_ID |
+-----------+-------------+-------------+
|     10002 |           2 |           3 |
|     10003 |           3 |           1 |
|     10004 |           4 |           1 |
+-----------+-------------+-------------+
3 rows in set (0.00 sec)
```

PART 4: BASIC SELECT QUERIES

1. **Display all repositories**

```
mysql> SELECT *
    -> FROM Repositories;
+---------+--------------+-----------------------------+------------------+------------+-------------+---------------------+-------------+--------------+
| Repo_ID | Name         | Description                 | Primary_Language | Visibility | Is_Archived | Created_On          | Stars_Count | Forks_Count |
+---------+--------------+-----------------------------+------------------+------------+-------------+---------------------+-------------+--------------+
|     101 | Smart-City-IoT | IoT analytics project     | C++              | public     |           0 | 2026-10-04 14:49:45 |          30 |            7 |
|     102 | DBMS-Lab     | Database management exercises | SQL              | public     |           0 | 2026-10-04 14:49:45 |          18 |            4 |
|     103 | ML-Toolkit   | Machine learning utilities  | Python           | public     |           0 | 2026-10-04 14:49:45 |          42 |           11 |
|     104 | Web-Portal   | Developer web portal        | JavaScript       | private    |           0 | 2026-10-04 14:49:45 |           9 |            2 |
|     105 | Legacy-App   | Archived legacy application | Java             | public     |           1 | 2026-10-04 14:49:45 |           5 |            1 |
+---------+--------------+-----------------------------+------------------+------------+-------------+---------------------+-------------+--------------+
5 rows in set (0.00 sec)
```

2. **Display only archived repositories**

```
mysql> SELECT Repo_ID, Name, Primary_Language, Is_Archived
    -> FROM Repositories
    -> WHERE Is_Archived = TRUE;
+---------+------------+------------------+-------------+
| Repo_ID | Name       | Primary_Language | Is_Archived |
+---------+------------+------------------+-------------+
|     105 | Legacy-App | Java             |           1 |
+---------+------------+------------------+-------------+
1 row in set (0.00 sec)
```

Figure 7.7: SQL query execution evidence, Review 2 page 7.

3. **Display repositories with more than 20 stars**

```
mysql> SELECT Repo_ID, Name, Stars_Count
    -> FROM Repositories
    -> WHERE Stars_Count > 20;
+---------+----------------+-------------+
| Repo_ID | Name           | Stars_Count |
+---------+----------------+-------------+
|     101 | Smart-City-IoT |          30 |
|     103 | ML-Toolkit     |          42 |
+---------+----------------+-------------+
2 rows in set (0.01 sec)
```

4. **Display repositories ordered by star count**

```
mysql> SELECT Name, Stars_Count, Forks_Count
    -> FROM Repositories
    -> ORDER BY Stars_Count DESC;
+----------------+-------------+-------------+
| Name           | Stars_Count | Forks_Count |
+----------------+-------------+-------------+
| ML-Toolkit     |          42 |          11 |
| Smart-City-IoT |          30 |           7 |
| DBMS-Lab       |          18 |           4 |
| Web-Portal     |           9 |           2 |
| Legacy-App     |           5 |           1 |
+----------------+-------------+-------------+
5 rows in set (0.00 sec)
```

5. **Display different repository languages**

```
mysql> SELECT DISTINCT Primary_Language
    -> FROM Repositories;
+------------------+
| Primary_Language |
+------------------+
| C++              |
| SQL              |
| Python           |
| JavaScript       |
| Java             |
+------------------+
5 rows in set (0.01 sec)
```

PART 5: AGGREGATE FUNCTIONS:

1. **Count total repositories**

```
mysql> SELECT COUNT(*) AS Total_Repositories
    -> FROM Repositories;
+--------------------+
| Total_Repositories |
+--------------------+
|                  5 |
+--------------------+
1 row in set (0.01 sec)
```

Figure 7.8: SQL query execution evidence, Review 2 page 8.

PART 8: JOINS:

1. **Repository + Collaborator**

```
mysql> SELECT
    ->     r.Repo_ID,
    ->     r.Name AS Repository_Name,
    ->     c.User_ID,
    ->     c.Role
    -> FROM Repositories r
    -> JOIN Collaborators c
    ->     ON r.Repo_ID = c.Repo_ID;
+---------+-----------------+---------+-------------+
| Repo_ID | Repository_Name | User_ID | Role        |
+---------+-----------------+---------+-------------+
|     101 | Smart-City-IoT  |       1 | Developer   |
|     102 | DBMS-Lab        |       1 | Developer   |
|     103 | ML-Toolkit      |       4 | Maintainer  |
+---------+-----------------+---------+-------------+
3 rows in set (0.00 sec)
```

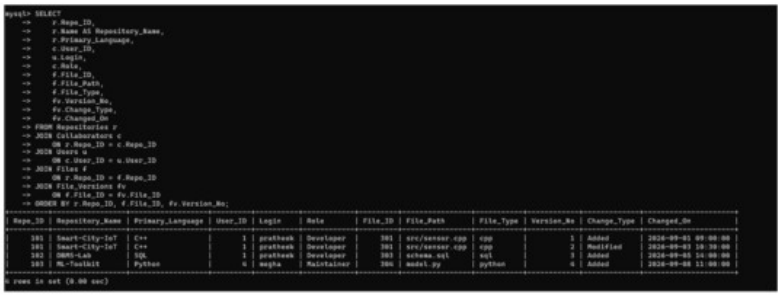
2. **Repository + User + Collaborator**

```
mysql> SELECT
    ->     r.Name AS Repository_Name,
    ->     u.Login,
    ->     u.Name AS User_Name,
    ->     c.Role
    -> FROM Repositories r
    -> JOIN Collaborators c
    ->     ON r.Repo_ID = c.Repo_ID
    -> JOIN Users u
    ->     ON c.User_ID = u.User_ID;
+-----------------+----------+-----------+-------------+
| Repository_Name | Login    | User_Name | Role        |
+-----------------+----------+-----------+-------------+
| Smart-City-IoT  | pratheek | Pratheek  | Developer   |
| DBMS-Lab        | pratheek | Pratheek  | Developer   |
| ML-Toolkit      | megha    | Megha     | Maintainer  |
+-----------------+----------+-----------+-------------+
3 rows in set (0.00 sec)
```

3. **Repository + Files**

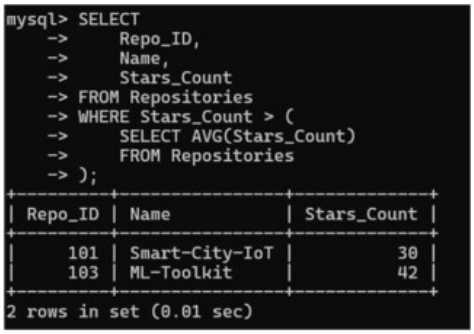
Figure 7.11: SQL query execution evidence, Review 2 page 11.

PART 9: MULTIPLE-TABLE JOIN:



PART 10: SUBQUERIES

1. **Repositories having above-average stars**



2. **Repositories with at least one fork**



Figure 7.14: SQL query execution evidence, Review 2 page 14.

7.5 NESTED QUERY

PART 12: SET OPERATIONS:

1. **Public or Private repositories**

```
mysql> SELECT
    ->     Repo_ID,
    ->     Name,
    ->     Visibility
    -> FROM Repositories
    -> WHERE Visibility = 'public'
    ->
    -> UNION
    ->
    -> SELECT
    ->     Repo_ID,
    ->     Name,
    ->     Visibility
    -> FROM Repositories
    -> WHERE Visibility = 'private';
+---------+----------------+------------+
| Repo_ID | Name           | Visibility |
+---------+----------------+------------+
|     101 | Smart-City-IoT | public     |
|     102 | DBMS-Lab       | public     |
|     103 | ML-Toolkit     | public     |
|     105 | Legacy-App     | public     |
|     104 | Web-Portal     | private    |
+---------+----------------+------------+
5 rows in set (0.00 sec)
```

PART 13: VIEW:

1. **Create repository information view**

```
mysql> CREATE VIEW Repository_Readiness AS
    -> SELECT
    ->     Repo_ID,
    ->     Name AS Repository_Name,
    ->     Primary_Language,
    ->     Visibility,
    ->     Is_Archived,
    ->     Stars_Count,
    ->     Forks_Count,
    ->     Created_On
    -> FROM Repositories;
Query OK, 0 rows affected (0.02 sec)
```

2. **Display the view**

```
mysql> SELECT *
    -> FROM Repository_Readiness;
+---------+-----------------+------------------+------------+-------------+-------------+-------------+---------------------+
| Repo_ID | Repository_Name | Primary_Language | Visibility | Is_Archived | Stars_Count | Forks_Count | Created_On          |
+---------+-----------------+------------------+------------+-------------+-------------+-------------+---------------------+
|     101 | Smart-City-IoT  | C++              | public     |           0 |          30 |           7 | 2026-10-04 14:49:45 |
|     102 | DBMS-Lab        | SQL              | public     |           0 |          18 |           4 | 2026-10-04 14:49:45 |
|     103 | ML-Toolkit      | Python           | public     |           0 |          42 |          11 | 2026-10-04 14:49:45 |
|     104 | Web-Portal      | JavaScript       | private    |           0 |           9 |           2 | 2026-10-04 14:49:45 |
|     105 | Legacy-App      | Java             | public     |           1 |           5 |           1 | 2026-10-04 14:49:45 |
+---------+-----------------+------------------+------------+-------------+-------------+-------------+---------------------+
5 rows in set (0.01 sec)
```

Figure 7.16: SQL query execution evidence, Review 2 page 16.

3. **View only active repositories**

```
mysql> SELECT *
    -> FROM Repository_Readiness
    -> WHERE Is_Archived = FALSE;
+---------+-----------------+------------------+------------+-------------+-------------+-------------+---------------------+
| Repo_ID | Repository_Name | Primary_Language | Visibility | Is_Archived | Stars_Count | Forks_Count | Created_On          |
+---------+-----------------+------------------+------------+-------------+-------------+-------------+---------------------+
|     101 | Smart-City-IoT  | C++              | public     |           0 |          30 |           7 | 2026-10-04 14:49:45 |
|     102 | DBMS-Lab        | SQL              | public     |           0 |          18 |           4 | 2026-10-04 14:49:45 |
|     103 | ML-Toolkit      | Python           | public     |           0 |          42 |          11 | 2026-10-04 14:49:45 |
|     104 | Web-Portal      | JavaScript       | private    |           0 |           9 |           2 | 2026-10-04 14:49:45 |
+---------+-----------------+------------------+------------+-------------+-------------+-------------+---------------------+
4 rows in set (0.00 sec)
```

PART 14: INDEXES:

1. **Frequently searched repository visibility**

```
mysql> CREATE INDEX idx_repository_visibility
    -> ON Repositories(Visibility);
Query OK, 0 rows affected (0.09 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

2. **Frequently searched repository language**

```
mysql> CREATE INDEX idx_repository_language
    -> ON Repositories(Primary_Language);
Query OK, 0 rows affected (0.14 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

3. **Commit lookup by repository**

```
mysql> CREATE INDEX idx_commit_repo
    -> ON Commits(Repo_ID);
Query OK, 0 rows affected (0.07 sec)
Records: 0  Duplicates: 0  Warnings: 0
```

4. **Pull request lookup by status**

PART 15: TRANSACTION:

```
mysql> START TRANSACTION;
Query OK, 0 rows affected (0.00 sec)

mysql> INSERT INTO Stars
    -> (Star_ID, Repo_ID, User_ID, Starred_On)
    -> VALUES
    -> (1000, 101, 5, NOW());
Query OK, 1 row affected (0.01 sec)

mysql> UPDATE Repositories
    -> SET Stars_Count = Stars_Count + 1
    -> WHERE Repo_ID = 101;
Query OK, 1 row affected (0.00 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> COMMIT;
Query OK, 0 rows affected (0.01 sec)

mysql> ROLLBACK;
Query OK, 0 rows affected (0.00 sec)
```

If something goes wrong — ROLLBACK

Figure 7.17: SQL query execution evidence, Review 2 page 17.

8. STORED PROCEDURES, FUCNTIONS AND TRIGGERS

8.1 TRIGGERS

PART 16: TRIGGER:

```
mysql> DELIMITER $$
mysql>
mysql> CREATE TRIGGER trg_merged_pr_repository
    -> AFTER INSERT ON Pull_Requests
    -> FOR EACH ROW
    -> BEGIN
    ->     IF NEW.Status = 'Merged' THEN
    ->         UPDATE Repositories
    ->         SET Is_Archived = FALSE
    ->         WHERE Repo_ID = NEW.Repo_ID;
    ->     END IF;
    -> END$$
Query OK, 0 rows affected (0.02 sec)

mysql>
mysql> DELIMITER ;
mysql> UPDATE Repositories
    -> SET Is_Archived = TRUE
    -> WHERE Repo_ID = 105;
Query OK, 0 rows affected (0.00 sec)
Rows matched: 1  Changed: 0  Warnings: 0

mysql> INSERT INTO Pull_Requests
    -> (PR_ID, Repo_ID, User_ID, Title, PR_Type, Status,
    ->  Created_On, Opened_By, Written_By)
    -> VALUES
    -> (704, 105, 1, 'Restore legacy application',
    ->  'Fix', 'Merged', NOW(), 1, 1);
Query OK, 1 row affected (0.12 sec)
```

```
mysql> SELECT Repo_ID, Name, Is_Archived
    -> FROM Repositories
    -> WHERE Repo_ID = 105;
+---------+------------+-------------+
| Repo_ID | Name       | Is_Archived |
+---------+------------+-------------+
|     105 | Legacy-App |           0 |
+---------+------------+-------------+
1 row in set (0.00 sec)
```

Figure 8.18: Trigger/function/procedure execution evidence, Review 2 page 18.

8.1 STORED PROCEDURES OR FUNCTIONS



PART 17: STORED FUNCTION

```
mysql> DROP FUNCTION IF EXISTS GetCommitCount;
Query OK, 0 rows affected, 1 warning (0.01 sec)

mysql> DELIMITER $$
mysql>
mysql> CREATE FUNCTION GetCommitCount(repo_id_input INT)
    -> RETURNS INT
    -> DETERMINISTIC
    -> READS SQL DATA
    -> BEGIN
    ->     DECLARE total_commits INT;
    ->
    ->     SELECT COUNT(*)
    ->     INTO total_commits
    ->     FROM Commits
    ->     WHERE Repo_ID = repo_id_input;
    ->
    ->     RETURN total_commits;
    -> END$$
Query OK, 0 rows affected (0.01 sec)

mysql>
mysql> DELIMITER ;
mysql> SELECT GetCommitCount(101) AS Total_Commits;
+---------------+
| Total_Commits |
+---------------+
|             2 |
+---------------+
1 row in set (0.01 sec)

mysql> SELECT
    ->     Repo_ID,
    ->     Name,
    ->     GetCommitCount(Repo_ID) AS Total_Commits
    -> FROM Repositories;
+---------+----------------+---------------+
| Repo_ID | Name           | Total_Commits |
+---------+----------------+---------------+
|     101 | Smart-City-IoT |             2 |
|     102 | DBMS-Lab       |             2 |
|     103 | ML-Toolkit     |             2 |
|     104 | Web-Portal     |             1 |
|     105 | Legacy-App     |             1 |
+---------+----------------+---------------+
5 rows in set (0.01 sec)
```

Figure 8.19: Trigger/function/procedure execution evidence, Review 2 page 19.

18: STORED PROCEDURE:

```
mysql> DELIMITER $$
mysql>
mysql> CREATE PROCEDURE UpdateRepositoryStatus(
    ->     IN repo_id_input INT,
    ->     IN archive_status BOOLEAN
    -> )
    -> BEGIN
    ->     UPDATE Repositories
    ->     SET Is_Archived = archive_status
    ->     WHERE Repo_ID = repo_id_input;
    -> END$$
Query OK, 0 rows affected (0.11 sec)

mysql>
mysql> DELIMITER ;
mysql> CALL UpdateRepositoryStatus(101, TRUE);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT
    ->     Repo_ID,
    ->     Name,
    ->     Is_Archived
    -> FROM Repositories
    -> WHERE Repo_ID = 101;
+---------+----------------+-------------+
| Repo_ID | Name           | Is_Archived |
+---------+----------------+-------------+
|     101 | Smart-City-IoT |           1 |
+---------+----------------+-------------+
1 row in set (0.00 sec)

mysql> CALL UpdateRepositoryStatus(101, FALSE);
Query OK, 1 row affected (0.01 sec)

mysql> SELECT
    ->     Repo_ID,
    ->     Name,
    ->     Is_Archived
    -> FROM Repositories
    -> WHERE Repo_ID = 101;
+---------+----------------+-------------+
| Repo_ID | Name           | Is_Archived |
+---------+----------------+-------------+
|     101 | Smart-City-IoT |           0 |
+---------+----------------+-------------+
1 row in set (0.00 sec)
```

Figure 8.20: Trigger/function/procedure execution evidence, Review 2 page 20.

9. FRONT-END DEVELOPMENT

Orbit is a GitHub-inspired web application built with Next.js and React. Users can sign in with GitHub or Google, or create an Orbit username and password account. After sign-in, users authorize the GitHub account they want Orbit to analyze. Orbit does not ask for GitHub passwords.

The workspace provides repository browsing and management, issues, pull requests, projects, organization repositories, contribution and language analytics, repository health checks, AI-assisted repository explanations, and an ecosystem map connecting repositories, contributors, languages and topics. The interface supports dark and light themes.

9.1 LOGIN AND SIGN-UP INTERFACE

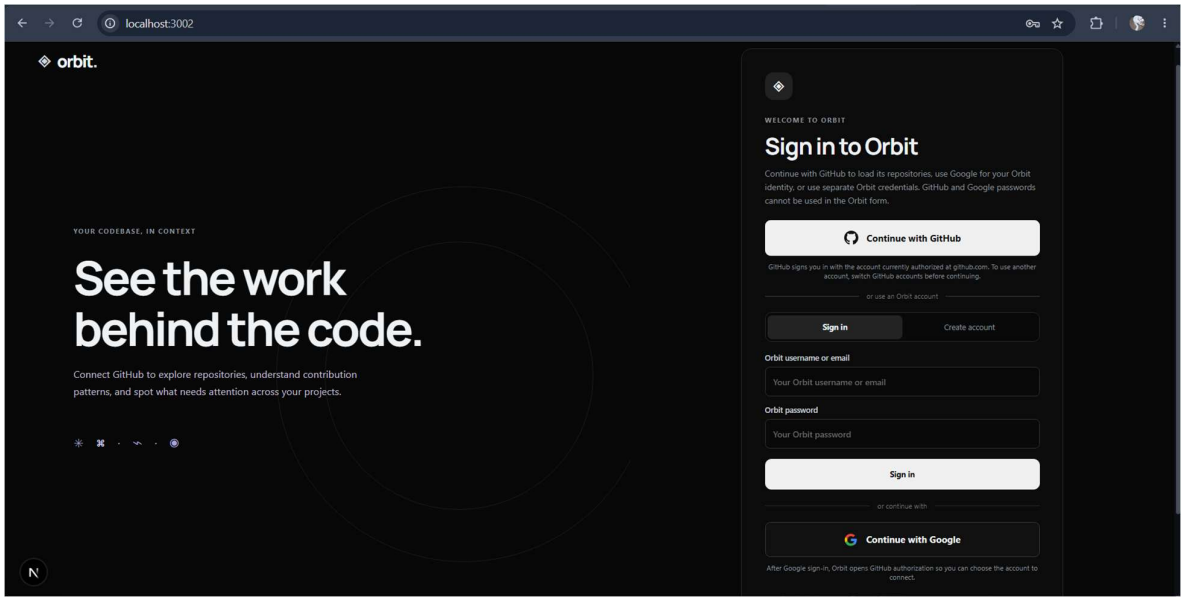
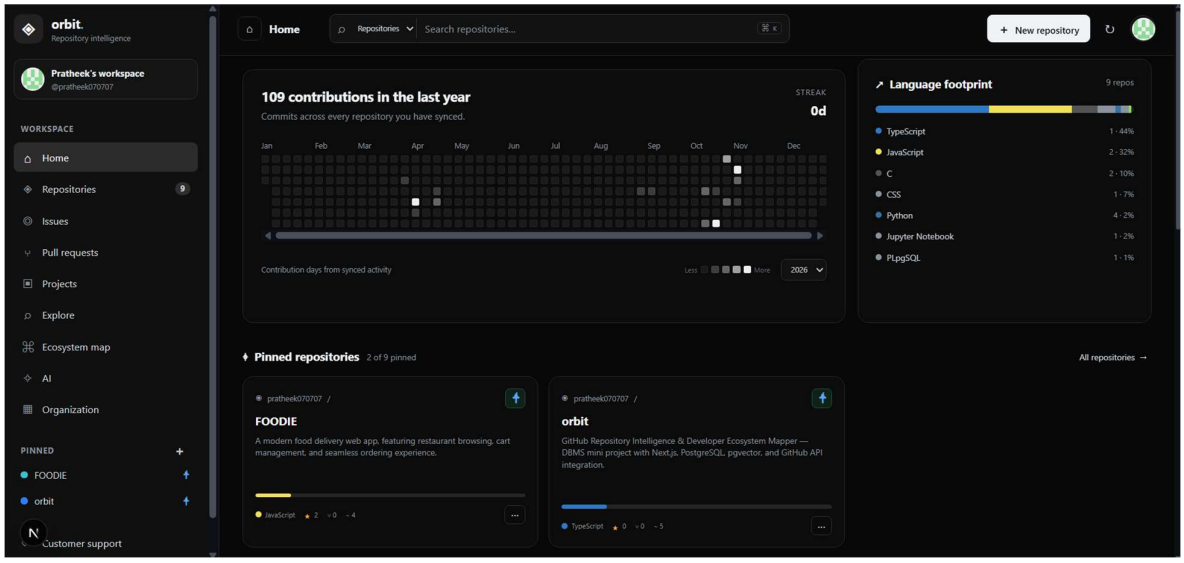


Figure 9.1: Orbit login and account registration interface.

9.2 MAIN DASHBOARD



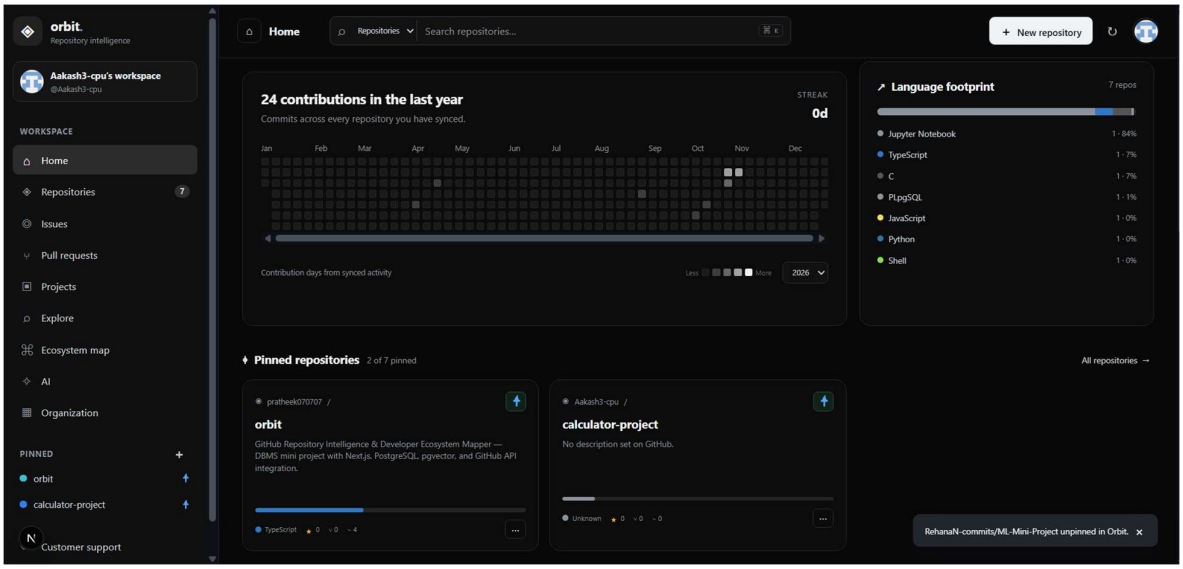


Figure 9.2: Main dashboard showing repository overview and language statistics.

9.3 REPOSITORY MANAGEMENT

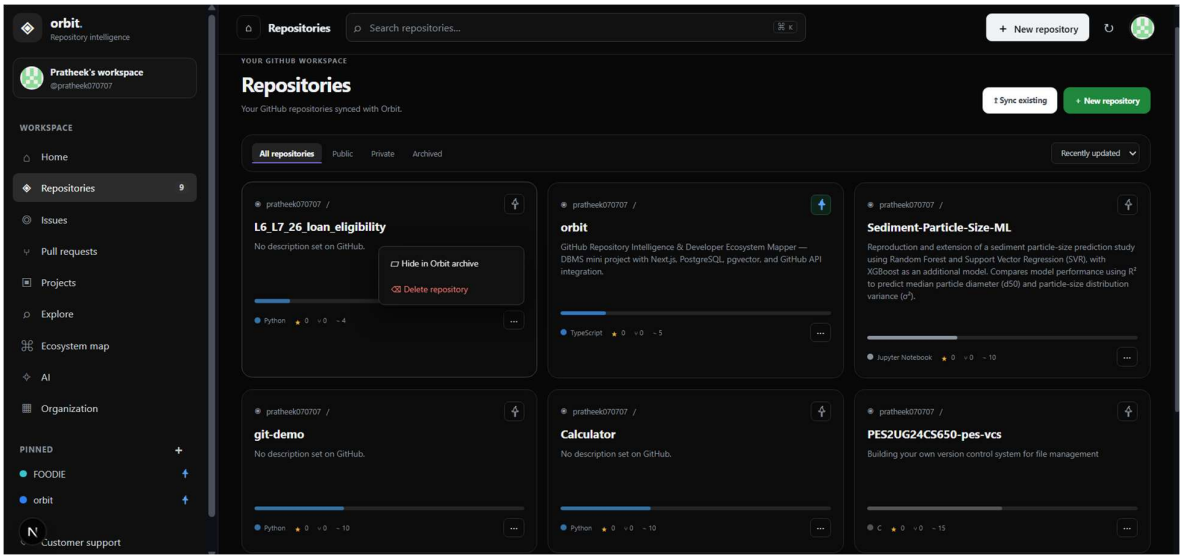


Figure 9.3: Repository management page displaying the user's repositories.

9.4 ISSUES MANAGEMENT

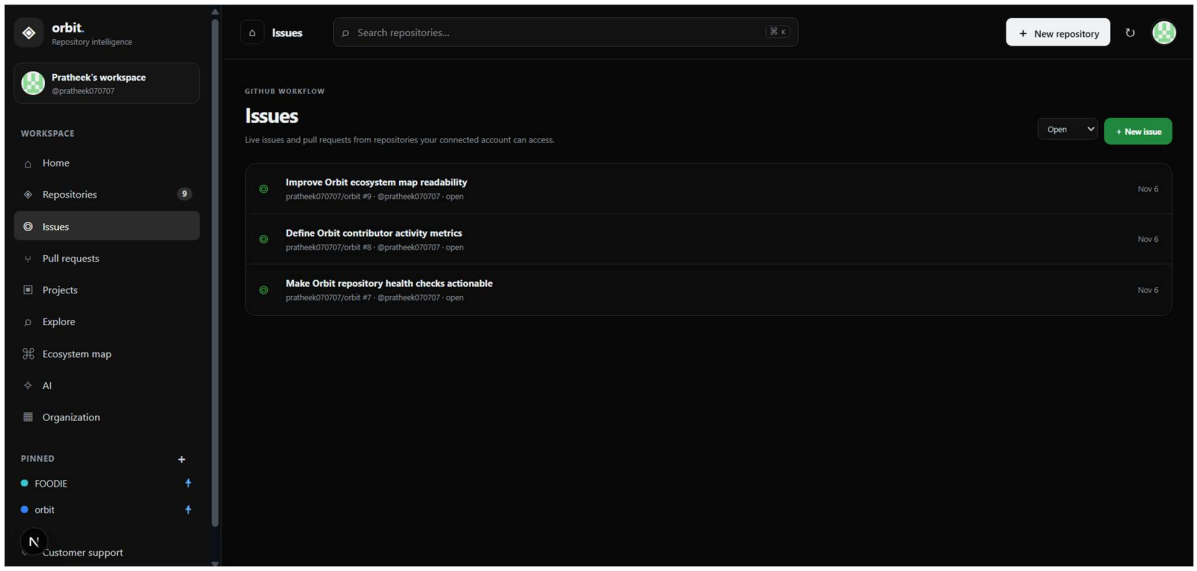
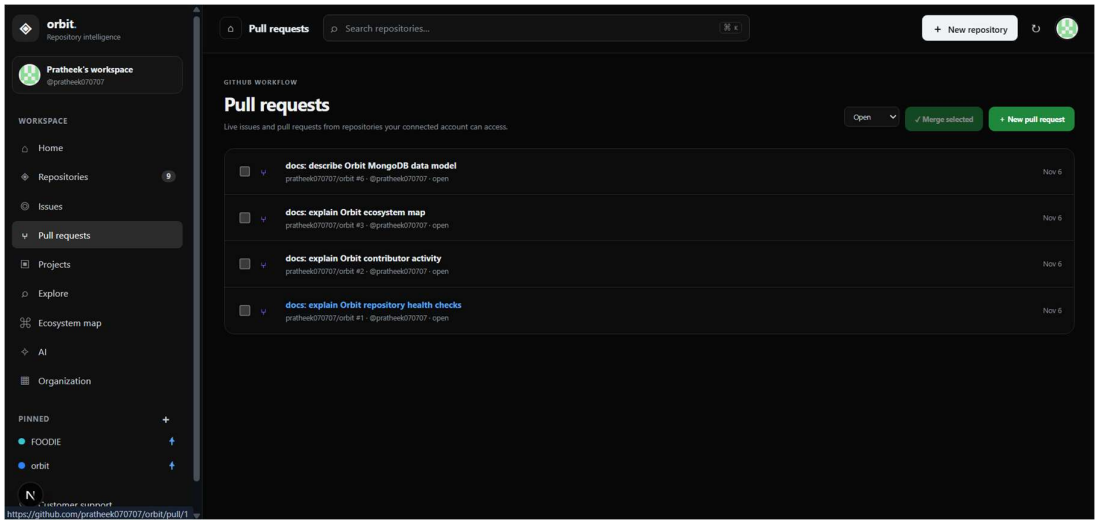


Figure 9.4: Issue management page displaying the user's repositories.

9.5 PR MANAGEMENT



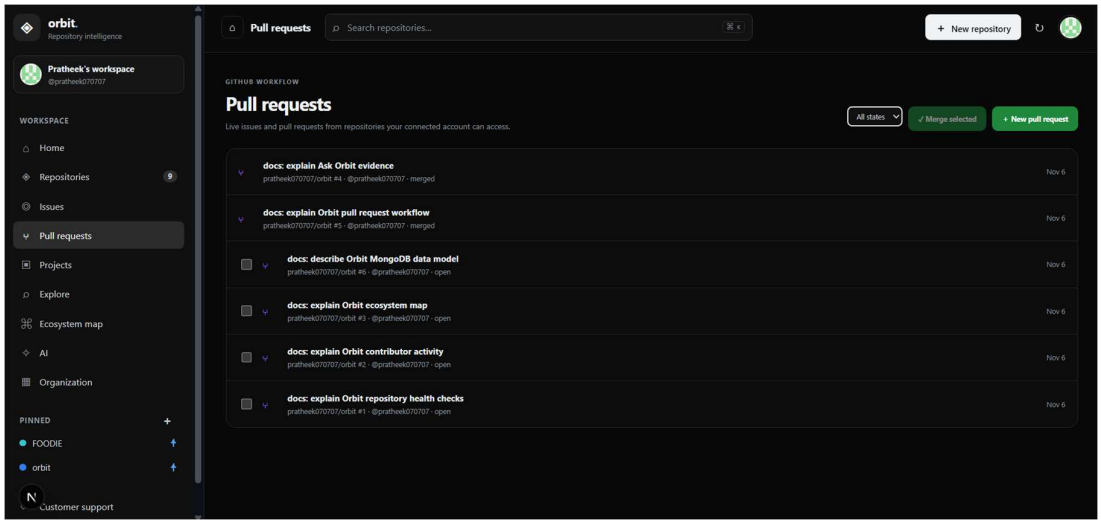


Figure 9.4: Pull request management page displaying the user's pr.

9.5 PROJECT MANAGEMENT

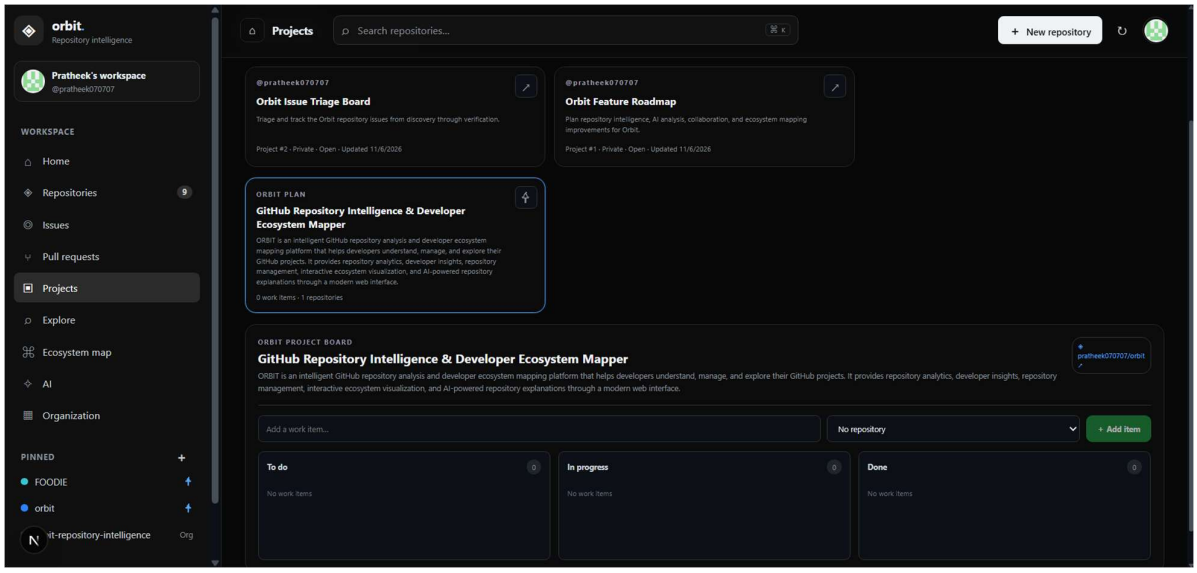


Figure 9.5: Project management page displaying the user's project from the repositories.

9.6 EXPLORE REPOSITORIES

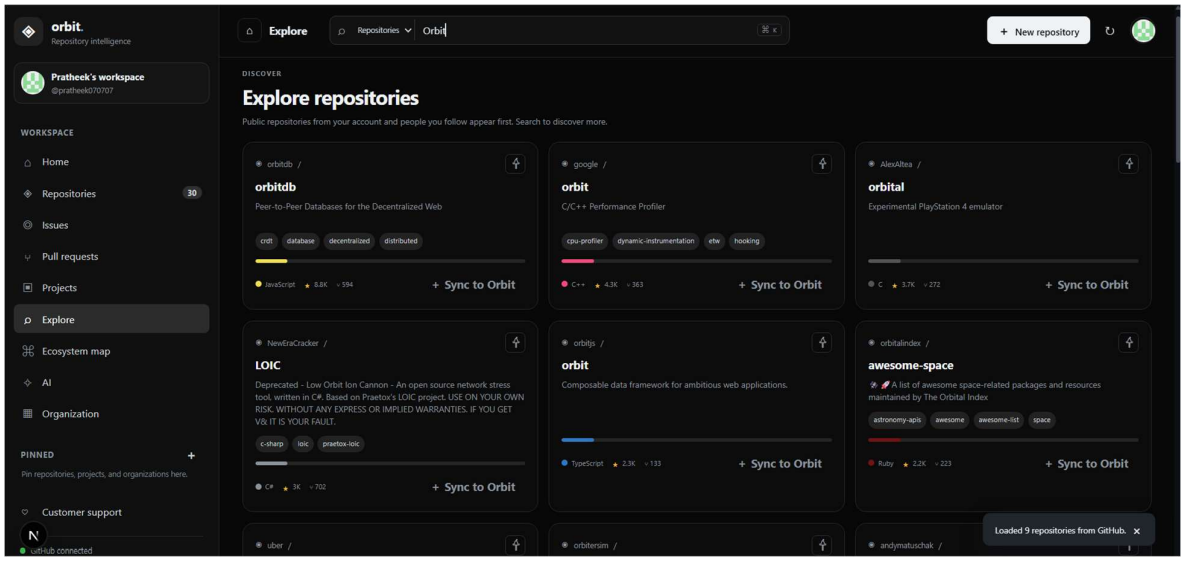


Figure 9.6: Explore repositories page for browsing available repositories.

9.6 EXPLORE USERS

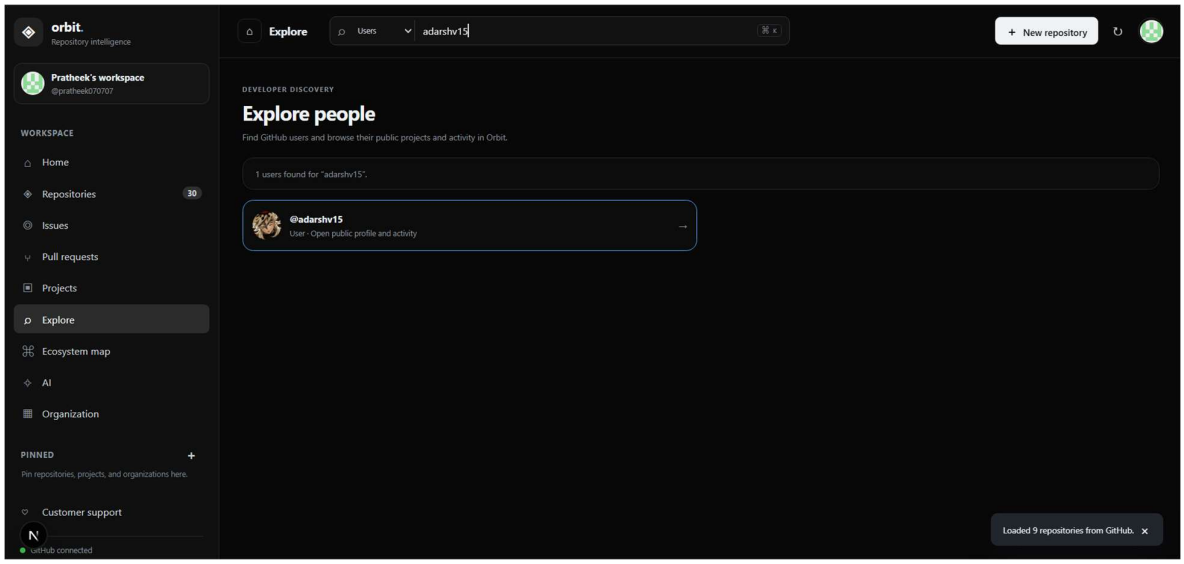


Figure 9.6: Explore people page for discovering developers.

9.7 DEVELOPER ECOSYSTEM MAP

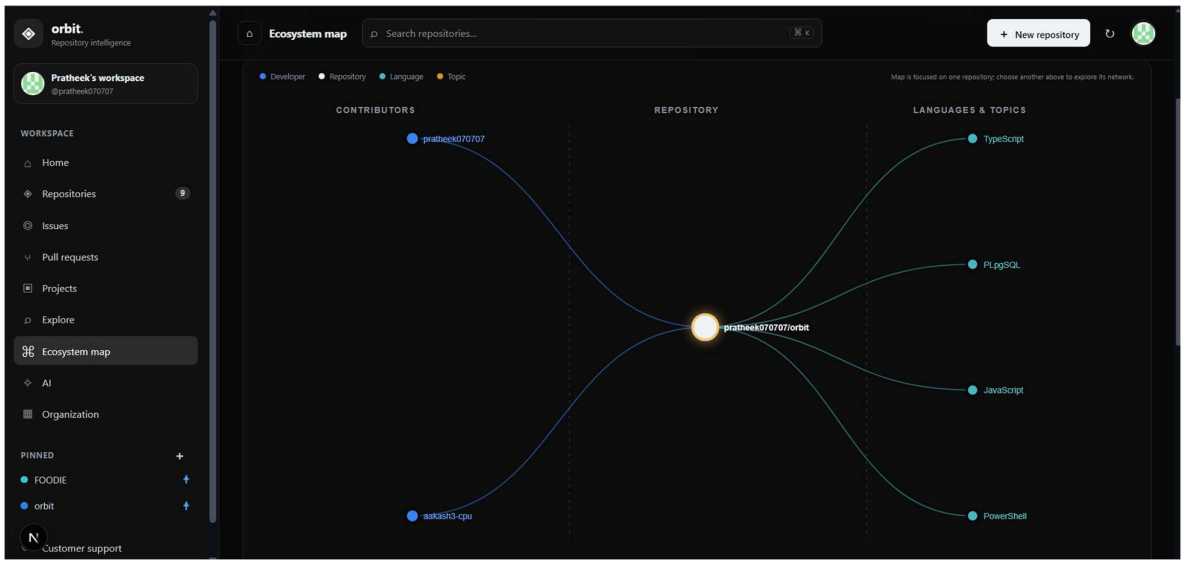
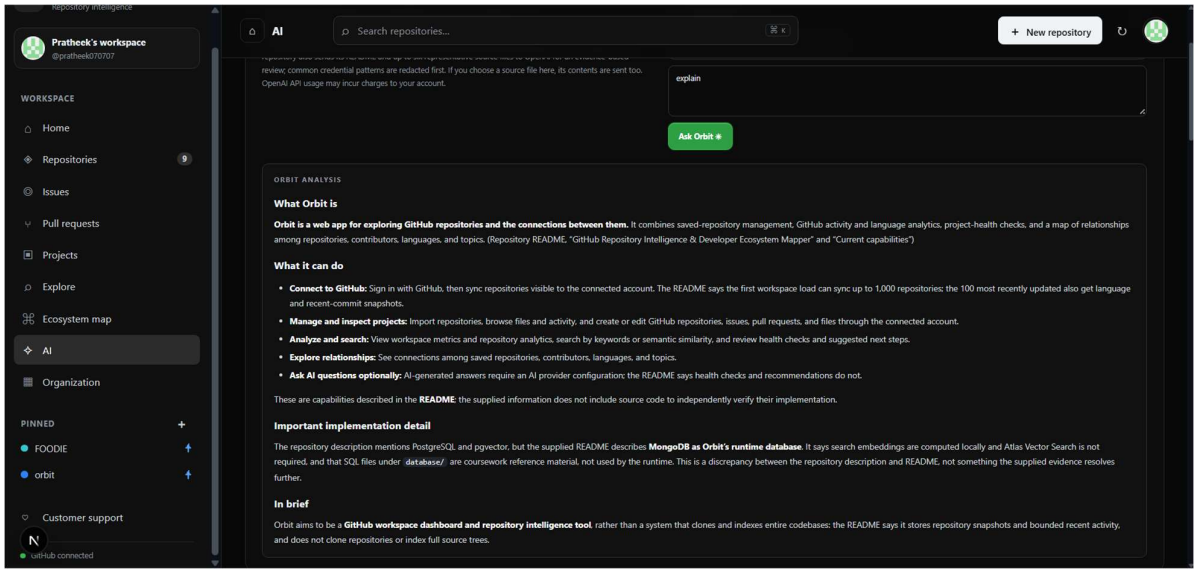


Figure 9.7: Ecosystem map connecting repositories, contributors, languages and topics.

9.8 AI-ASSISTED REPOSITORY INSIGHTS



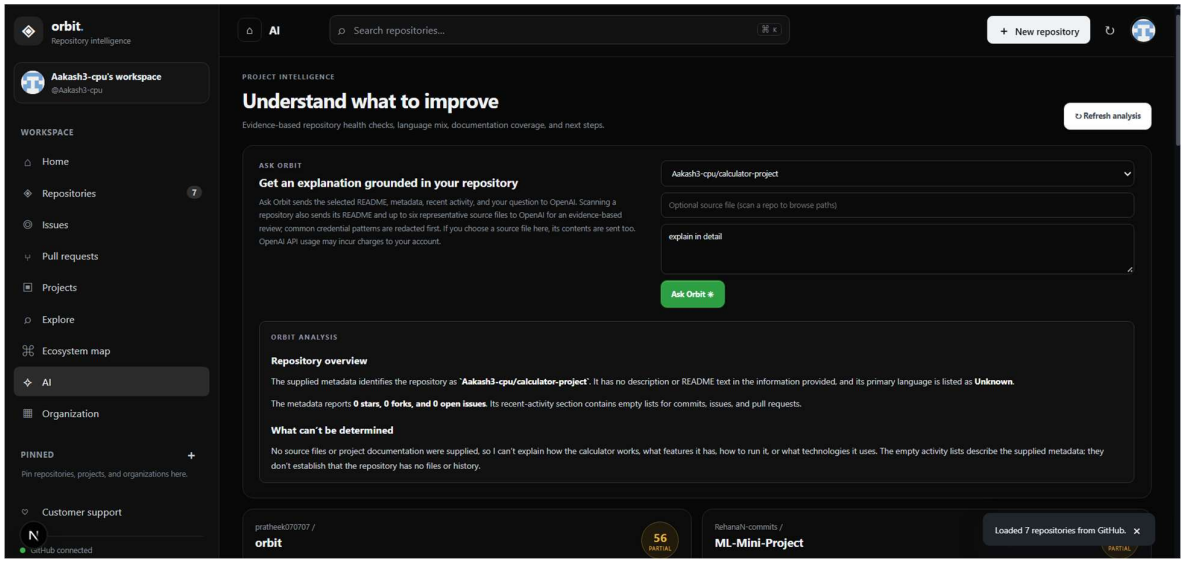


Figure 9.8: AI-assisted repository analysis and improvement recommendations.

9.9 ORGANIZATION

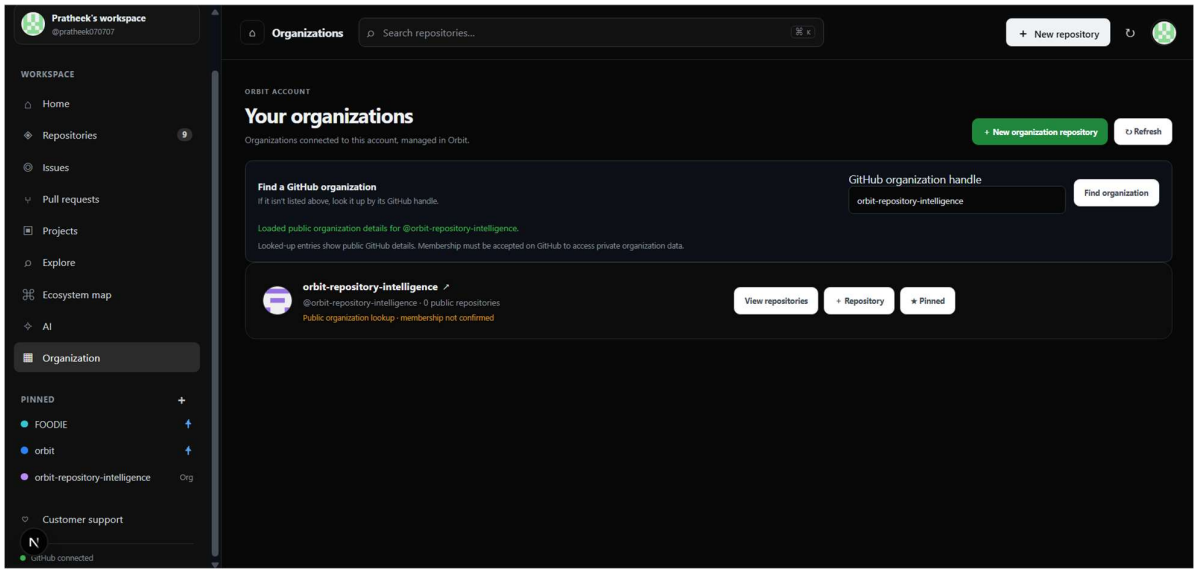


Figure 9.9: Organization page displaying the user's organization details.

9.10 CUSTOMER SUPPORT

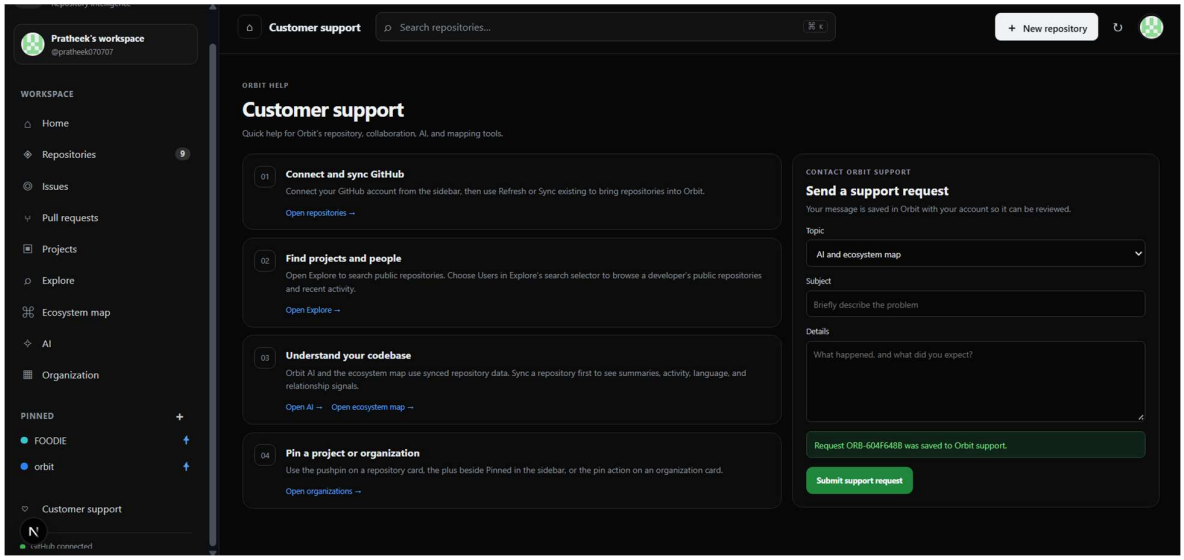


Figure 9.9: Customer support page with help topics and support request form.

9.11 USER PROFILE

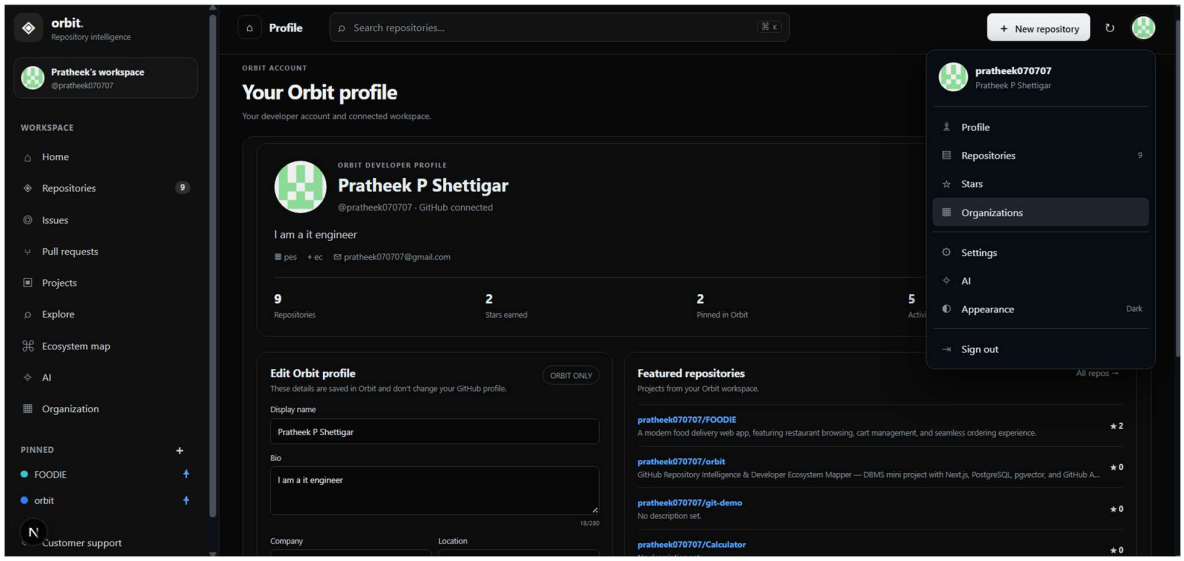


Figure 9.11: User profile page showing account and developer information.

9.12 APPLICATION SETTINGS

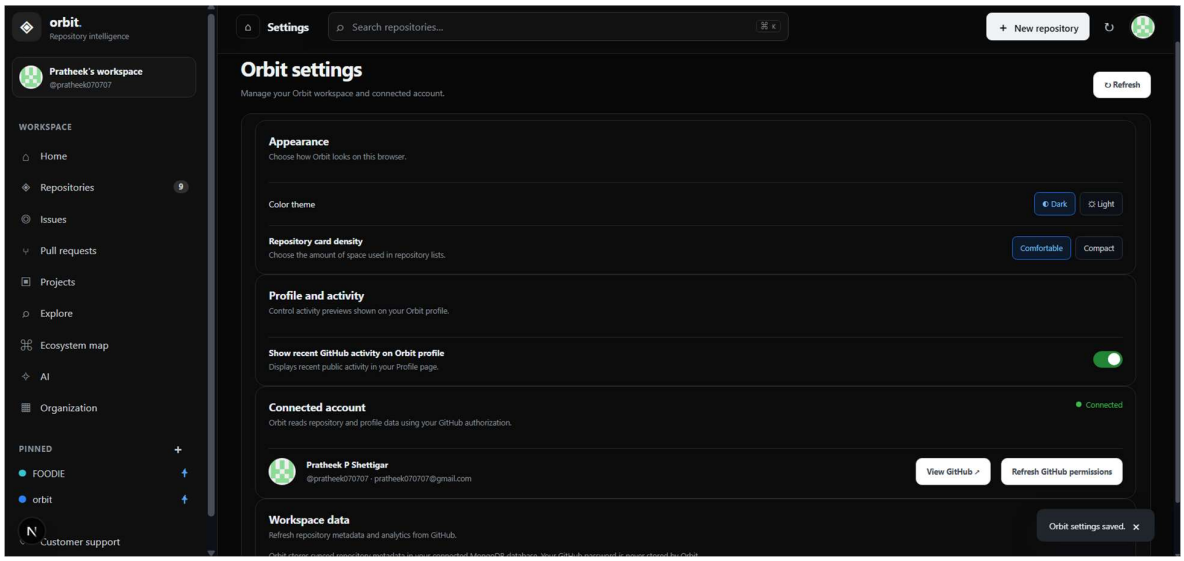


Figure 9.10: Orbit settings page for appearance, preferences and account options.

9.13 LIGHT-THEME DASHBOARD

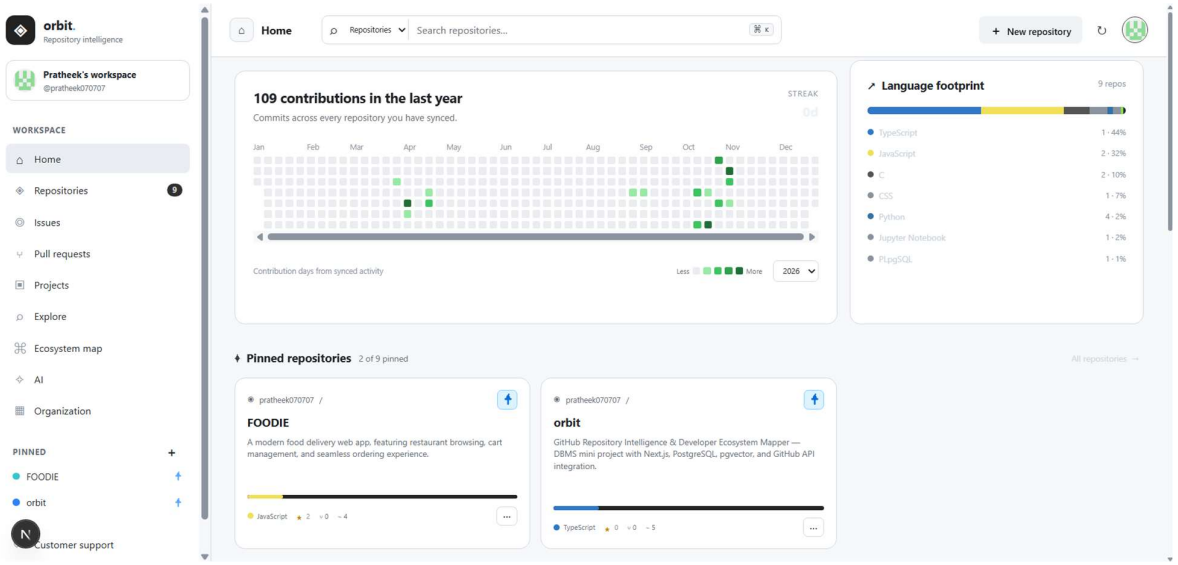
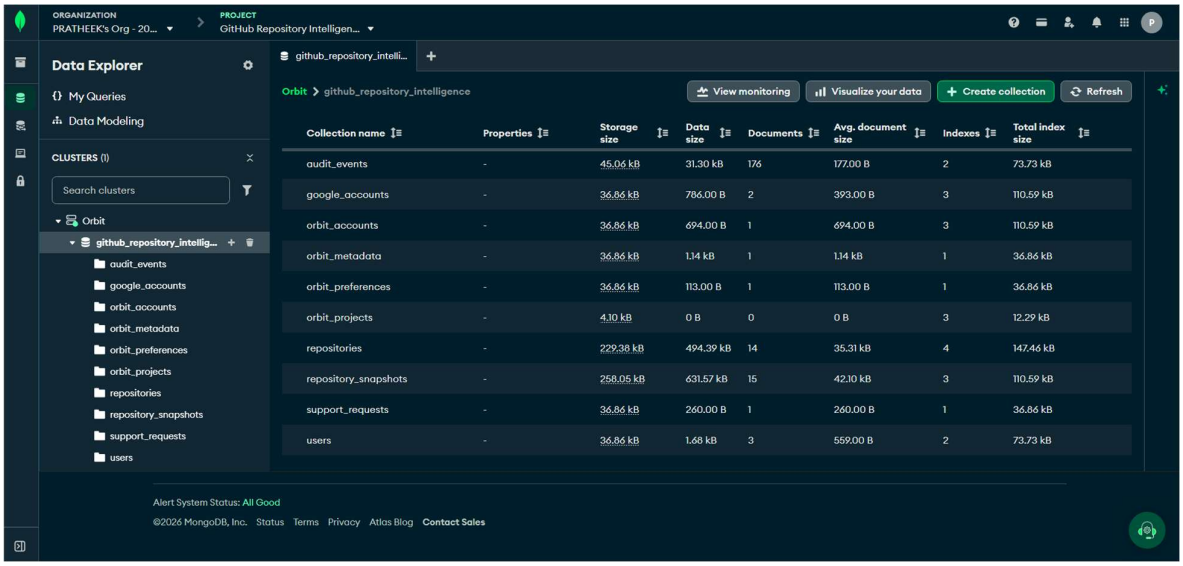


Figure 9.13: Main dashboard displayed in light theme.

10. MongoDB DATABASE

Orbit uses MongoDB as its runtime NoSQL document database. The Next.js backend connects through the MongoDB driver using MONGODB_URI and MONGODB_DATABASE environment variables. The Atlas Data Explorer screenshot supplied with the project shows collections for account identities, repositories, repository snapshots, preferences, projects, metadata, audit events and support requests. MongoDB stores application records; GitHub remains the source for GitHub-hosted repository activity.



10.1 MongoDB collections

The collections below separate Orbit account data, synced GitHub information and application activity. MongoDB documents can keep related repository details together, while indexes support common lookups and prevent duplicate identities.

Collection	Purpose in Orbit
users	Orbit workspace profiles and linked provider identity. GitHub access tokens are encrypted before storage.
orbit_accounts	Optional Orbit username accounts. Passwords are stored as salted password hashes, not plaintext.
google_accounts	Google identity records linked to an Orbit workspace account.
repositories	Synced repository metadata, ownership, visibility, language, activity counters and pins.
repository_snapshots	Repository README, language breakdown, analytics, recent activity and scan snapshots.
orbit_projects	User-created Orbit projects and their repository/task items.
orbit_preferences	Per-user interface and workspace preferences.
orbit_metadata	Orbit application metadata, objectives and schema version.
audit_events	Recorded account and repository actions for activity history.
support_requests	Support requests submitted through Orbit.

Table 10.1: MongoDB collections and their roles in Orbit.

10.2 Account storage and security

Orbit username/password sign-up stores a salted password hash and salt in orbit_accounts; the submitted password itself is not stored. Google identities are represented in google_accounts, while users holds the Orbit workspace profile and linked GitHub identity. GitHub access tokens are encrypted by the application before being persisted. OAuth providers verify their own credentials; Orbit does not collect a GitHub password.

10.3 MongoDB and the SQL coursework

MySQL is used for the academic relational schema and SQL demonstrations in this report, including joins, aggregate queries, updates, deletes, triggers and stored routines. MongoDB is the live application backend for Orbit. These are complementary parts of the project: the MySQL exercises demonstrate relational DBMS concepts, while MongoDB supports the website’s document-oriented runtime data.

REFERENCES

[1] DBMS Review 1 submission: ER diagram, relational schema and DDL statements, supplied project document.

[2] DBMS Review 2 submission: SQL data manipulation, queries, joins, subqueries, views, indexes, transactions, triggers, stored routines, CTEs and window functions, supplied project document.

[3] MySQL Reference Manual, MySQL Documentation, https://dev.mysql.com/doc/ .

[4] Git documentation, https://git-scm.com/doc .

GitHub Link: https://github.com/pratheek070707/orbit